

Node.js Pending Bucket

Developed by fahadvn

June 14, 2025

Pending-pdf © 2025 | Developed by fahadvn - LinkedIn

1 Introduction

This document organizes pending Node.js topics into two categories: **Practical** (tasks requiring coding, configuration, or hands-on implementation) and **Theoretical** (concepts needing clarity or deeper understanding). The topics are sourced from the provided list and grouped by their nature to facilitate structured learning and revision.

2 Practical Pendings

This section lists tasks that involve coding, configuration, or practical implementation in Node.js.

2.1 File System Operations

- Create a server using Node.js HTTP module
- Write current date and time to a file using fs module
- Create a file using fs module if it doesn't exist, else read it
- Delete a file using fs.unlink
- Transfer data from one file to another using pipe
- fs.stat to check file properties
- Create an HTTP GET endpoint to read content from a file

2.2 Server and API Development

- Create a server using http.createServer
- Create a server using Express.js
- Create an HTTP GET endpoint
- Implement dynamic routing

- Set up router-level middleware
- Create middleware to block all POST requests
- Create middleware to block all DELETE requests
- Create middleware to validate a specific header
- Create an API endpoint to count the occurrence of a specific character in a string
- Create an API endpoint to add two numbers using path and query parameters
- Configure static files using `express.static`
- Render plain HTML
- Set up a view engine (e.g., EJS) and use `res.render`
- Implement router chaining
- Pass data to templates
- Handle form submission and process user input in a POST method

2.3 Middleware Implementation

- Create application-level middleware to block GET requests and log other incoming requests (e.g., PUT with current time)
- Create error-handling middleware
- Implement middleware to log names of every incoming parameter
- Set up third-party middleware (e.g., `morgan`, `multer`)
- Implement session management using `express-session`

2.4 Child Processes and Multithreading

- Create a worker thread
- Implement `child_process.fork()`
- Implement `child_process.spawn()`
- Implement `child_process.exec()`
- Set up clustering for scalability

2.5 Streams and Buffers

- Implement a transform stream
- Use pipe for stream operations
- Encrypt a string in Node.js

- Use the Buffer class for data manipulation

2.6 Authentication and Security

- Implement Passport.js authentication
- Set up JWT (JSON Web Tokens)
- Configure HTTP-only cookies
- Set cookie attributes (sameSite, maxAge, expires)
- Implement CSRF protection
- Configure CORS policy
- Sanitize input to prevent SQL injection and XSS

2.7 Other Practical Tasks

- Create a callback hell and convert it to promises
- Use promisify to convert callback-based functions to promises
- Parse a URL using the url module
- Set up environment variables in a .env file
- Configure nodemon for development
- Implement rate limiting for APIs
- Set up a cron job
- Create a custom module
- Toggle a checkbox by button click
- Display username on homepage

3 Theoretical Pendings

This section covers concepts requiring deeper understanding or clarity.

3.1 Node.js Fundamentals

- What is Node.js? Features, applications, and architecture
- Why use the V8 engine?
- Single-threaded vs. multi-threaded
- Synchronous vs. asynchronous execution
- Event-driven programming and architecture
- How Node.js handles concurrency

- Node.js scalability with clustering
- Global objects (`__dirname`, `__filename`, `module`, `process`)
- REPL (Read-Eval-Print-Loop)
- `process.nextTick` vs `setImmediate` vs `setTimeout`
- Microtask queue vs. macrotask queue
- Event loop phases and working
- Thread pool and libuv
- Concurrency vs. parallelism
- I/O-bound vs. CPU-bound operations
- Starvation in Node.js

3.2 Modules and Package Management

- `require` vs `import`
- CommonJS vs. ES Modules
- `package.json` vs `package-lock.json`
- `npm` vs `npx`
- `nvm` vs `npm`
- Global vs. local dependencies
- `devDependencies` vs. production dependencies
- Scaffolding in Node.js
- Purpose of `.npmrc`
- Core modules (e.g., `fs`, `http`, `os`, `path`, `url`)
- Third-party modules (e.g., `morgan`, `multer`)

3.3 HTTP and Networking

- HTTP vs. HTTPS
- Components of HTTP request and response
- HTTP methods (GET, POST, PUT, PATCH, DELETE, OPTIONS, HEAD, TRACE)
- Idempotency (e.g., PUT vs POST)
- HTTP status codes (e.g., 200, 201, 204, 400, 403, 404, 500)
- HTTP headers (`setHeader`, `writeHead`, `Content-Type`)
- Preflight requests and CORS
- Same-origin policy

- Statelessness of HTTP and mitigation with cookies/sessions
- Content negotiation
- TCP/IP and DNS
- MIME types
- HTTP/2
- OPTIONS method
- WebSockets

3.4 Middleware and Express.js

- What is middleware? Types (application-level, router-level, error-handling, built-in, third-party)
- `app.use` vs `app.set`
- `app.all` vs `app.get`
- `express.json` vs `express.urlencoded`
- `app.locals` and `res.locals`
- Middleware positioning and order
- Error-handling middleware arguments and invocation
- Router chaining
- Dynamic routing vs. static routing
- Query parameters vs. path parameters

3.5 Authentication and Security

- Authentication vs. authorization
- Session vs. cookies vs. `localStorage` vs. `sessionStorage`
- Disadvantages of cookies
- HTTP-only cookies, `sameSite`, and domains
- CSRF (Cross-Site Request Forgery)
- Content Security Policy (CSP)
- JWT (JSON Web Tokens)
- SSL/TLS
- Hashing vs. encryption

3.6 Streams and Events

- Streams and types (Readable, Writable, Duplex, Transform)
- Buffer class
- EventEmitter and custom events
- on vs once in EventEmitter
- Reactor pattern

3.7 Child Processes and Multithreading

- `child_process` module (`fork`, `spawn`, `exec`, `execFile`)
- Process vs. threads
- Worker threads
- Clustering vs. `child_process.fork`
- Exit codes

3.8 Other Concepts

- REST principles and API versioning
- MVC architecture
- View engines (e.g., EJS) and partials
- `promise.all` vs `promise.race`
- Error-first callbacks
- `JSON.parse` vs `JSON.stringify`
- Browser cache and types of browser storage
- `setInterval` vs `setTimeout`
- Command-line arguments
- User agent
- `zlib` and `util` modules
- `WeakSet` and `WeakMap`
- `StructuredClone` and `Object.entries`
- `Symbol` and `Array.from`
- Rate limiting of APIs
- Cron jobs
- Latest Node.js features

3.9 MongoDB-Specific Concepts

- Clustered collection
- Modify collection name
- \$expr
- \$elemMatch
- \$facet
- Multikey index vs. compound index
- Covered query
- Drawbacks of indexing

